Algoritmos de Ordenação em tempo linear

Estruturas de Dados

adalberto@computacao.ufcg.edu.br

Ordenação por comparação

Qual o melhor tempo dos algoritmos de ordenação vistos até agora?

É possível ser mais rápido usando ordenação por comparação? Por quê?

É possível ordenar sem usar comparações? Em que tempo?

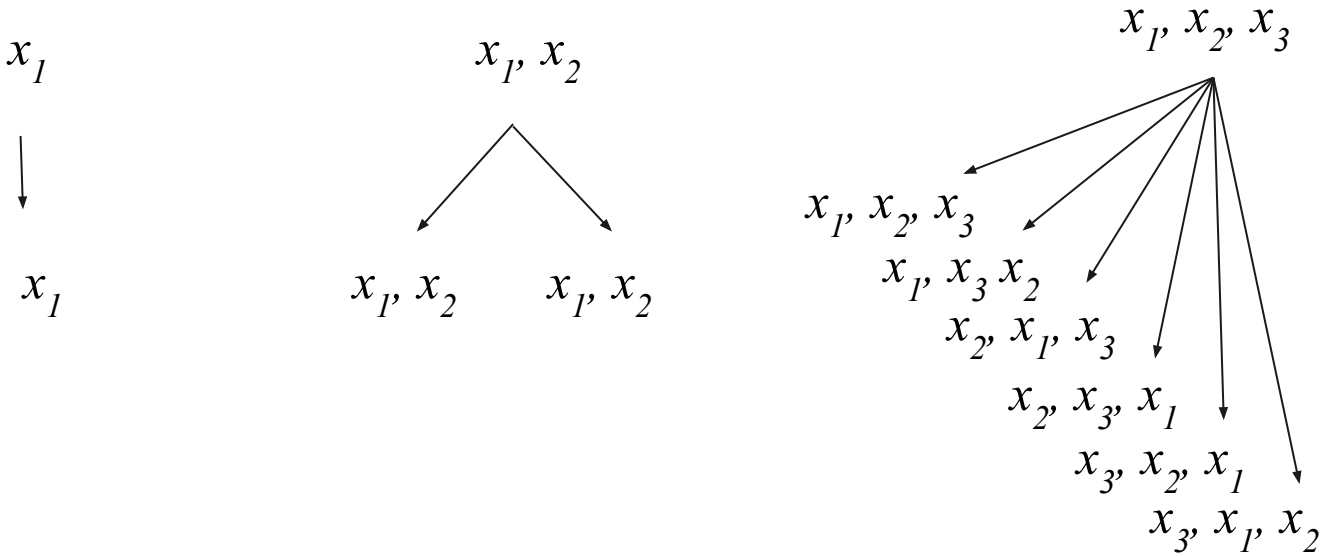
Ordenação por comparação

x_1

x_1, x_2

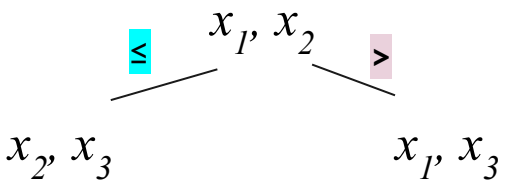
x_1, x_2, x_3

Ordenação por comparação



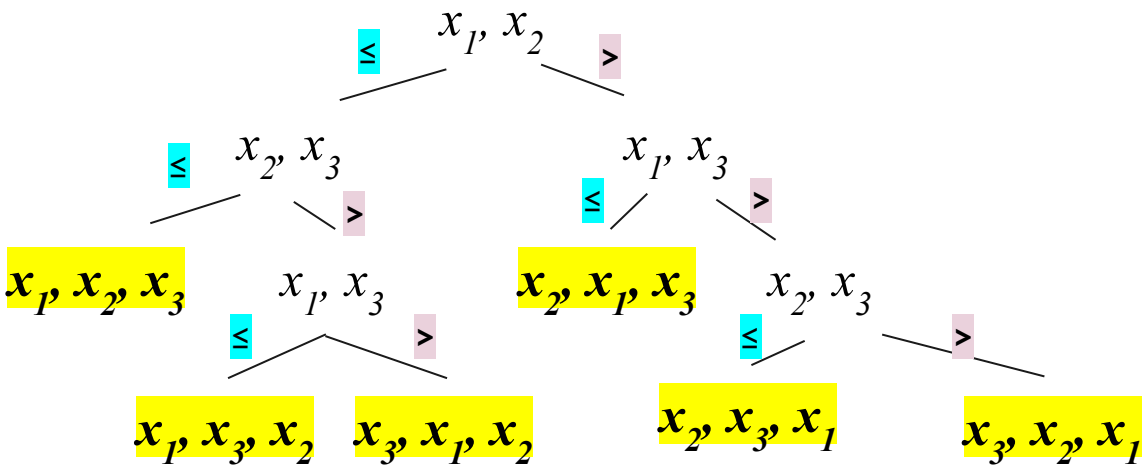
Ordenação por comparação (árvore de decisão)

x_1, x_2, x_3

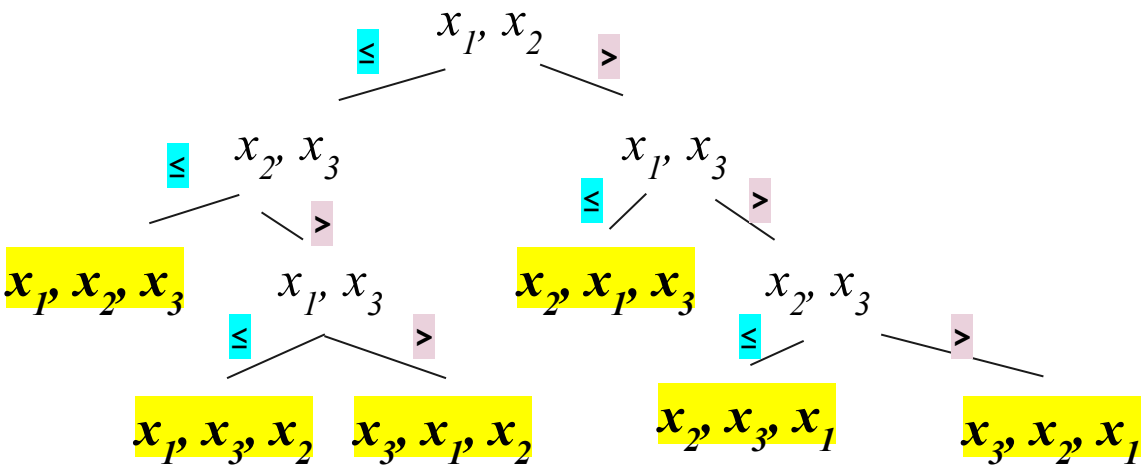


Ordenação por comparação (árvore de decisão)

x_1, x_2, x_3

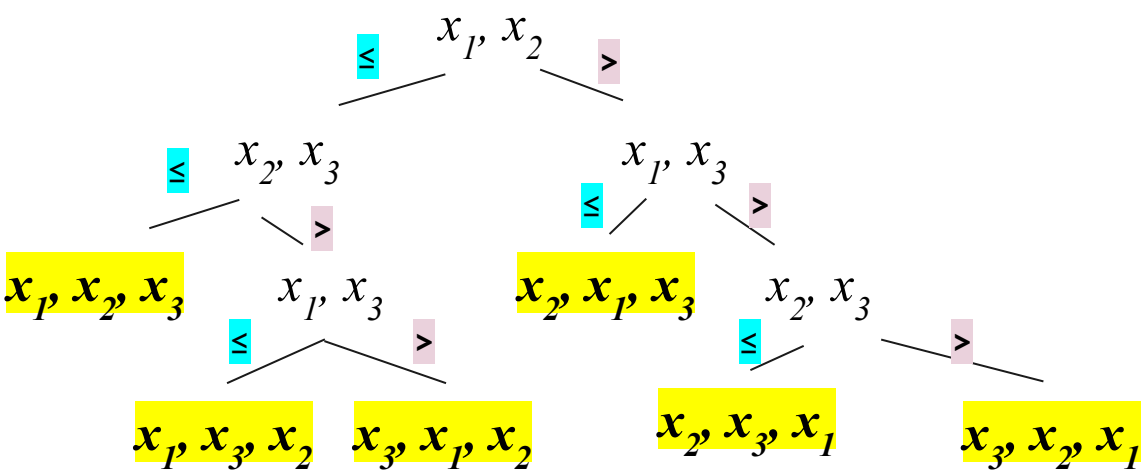


Ordenação por comparação (árvore de decisão)



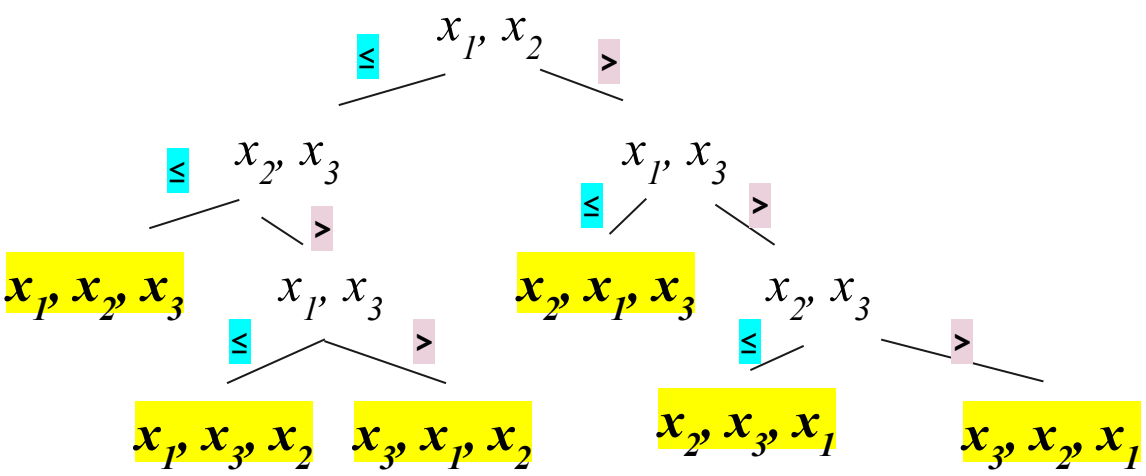
$$\#folhas \geq n!$$
$$\#folhas = 2^h$$

Ordenação por comparação (árvore de decisão)



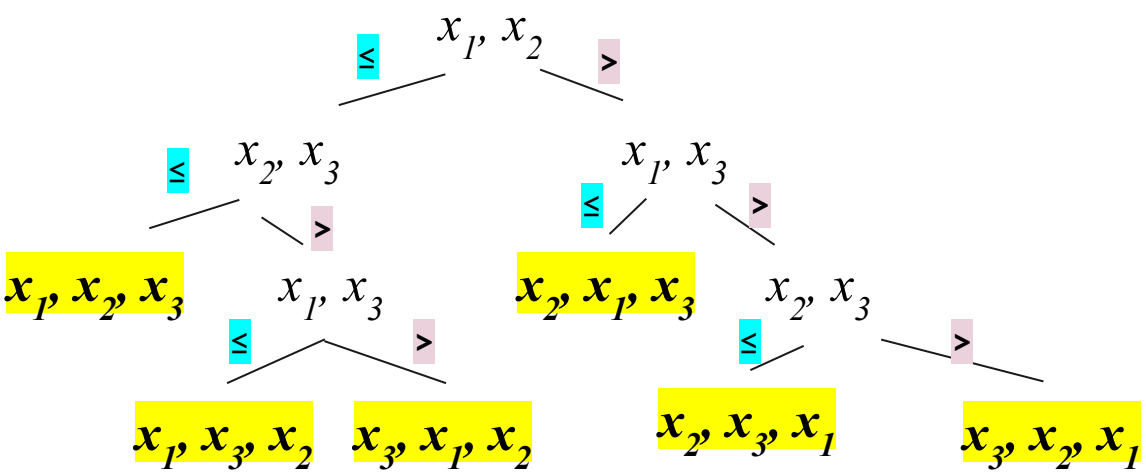
$$\#folhas \geq n!$$
$$\#folhas = 2^h$$
$$2^h \geq n!$$

Ordenação por comparação (árvore de decisão)



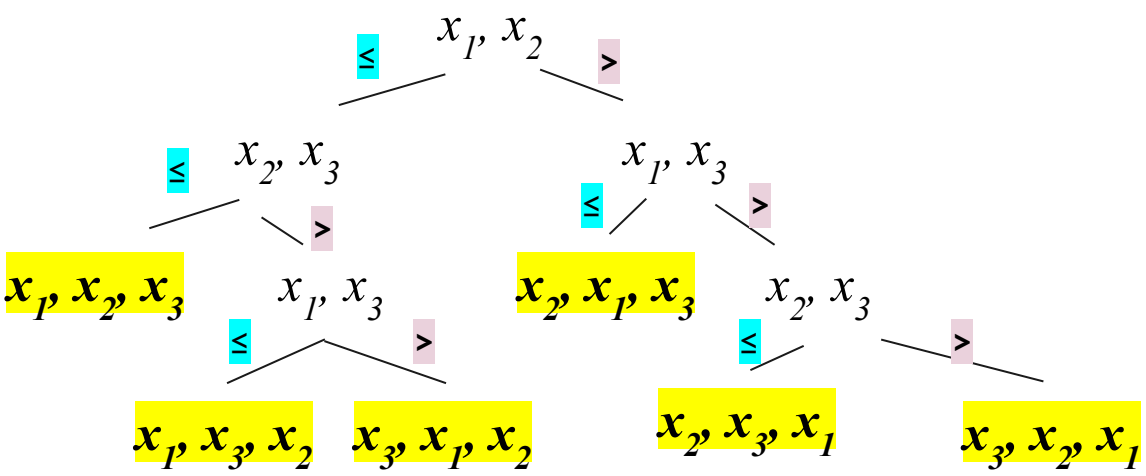
$$\#folhas \geq n!$$
$$\#folhas = 2^h$$
$$2^h \geq n!$$
$$\log 2^h \geq \log n!$$

Ordenação por comparação (árvore de decisão)



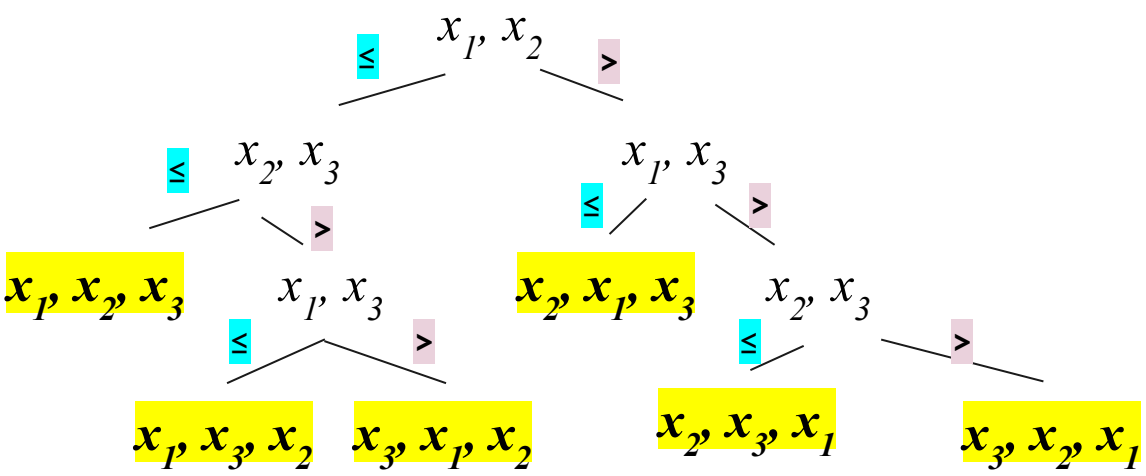
$$\#folhas \geq n!$$
$$\#folhas = 2^h$$
$$2^h \geq n!$$
$$\log 2^h \geq \log n!$$
$$h \geq \log(n/e)^n$$

Ordenação por comparação (árvore de decisão)



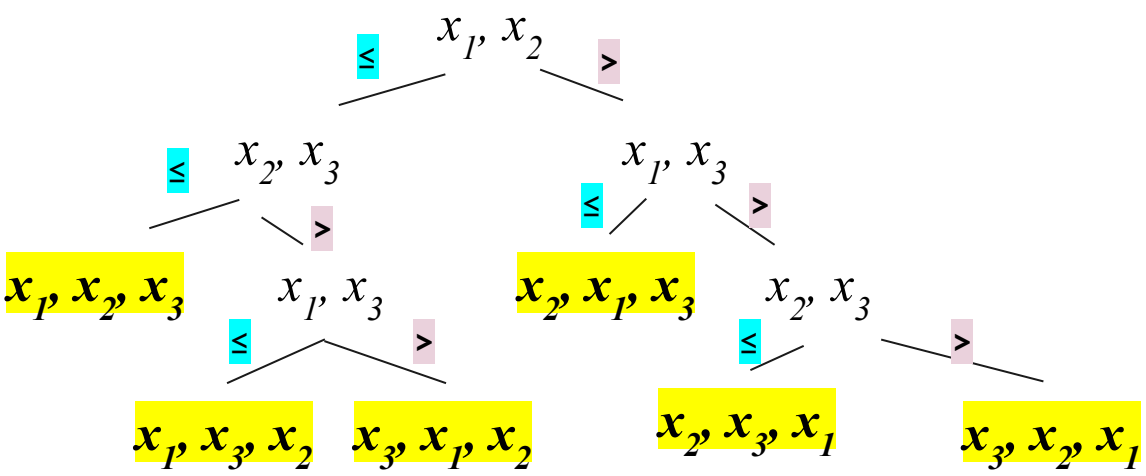
$$\#folhas \geq n!$$
$$\#folhas = 2^h$$
$$2^h \geq n!$$
$$\log 2^h \geq \log n!$$
$$h \geq \log(n/e)^n$$
$$h \geq n \log(n/e)$$

Ordenação por comparação (árvore de decisão)



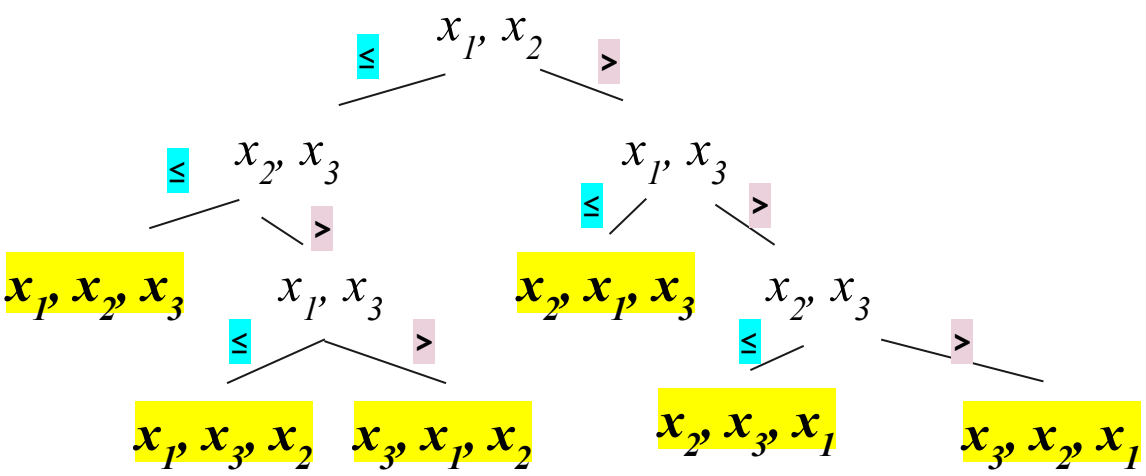
$$\#folhas \geq n!$$
$$\#folhas = 2^h$$
$$2^h \geq n!$$
$$\log 2^h \geq \log n!$$
$$h \geq \log(n/e)^n$$
$$h \geq n \log(n/e)$$
$$h \geq n(\log n - \log e)$$

Ordenação por comparação (árvore de decisão)



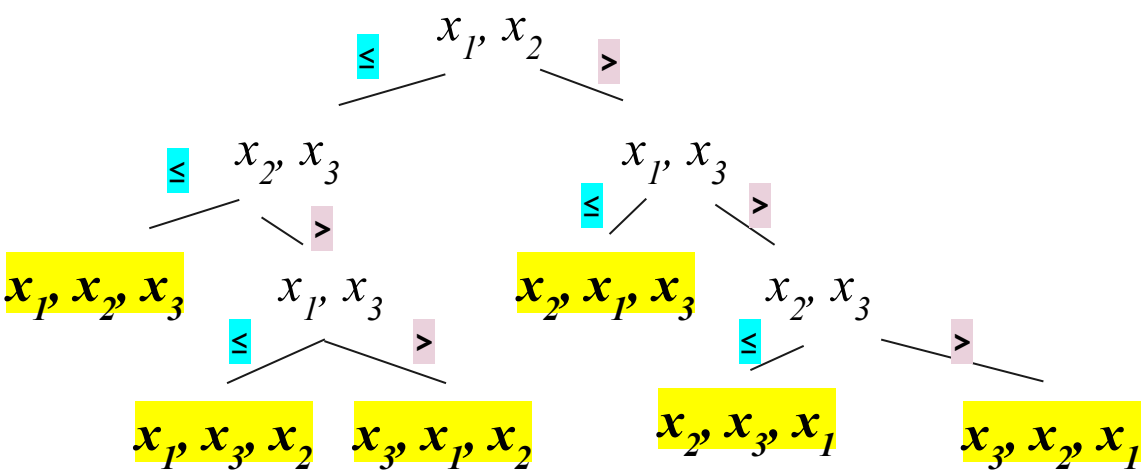
$$\#folhas \geq n!$$
$$\#folhas = 2^h$$
$$2^h \geq n!$$
$$\log 2^h \geq \log n!$$
$$h \geq \log(n/e)^n$$
$$h \geq n \log(n/e)$$
$$h \geq n(\log n - \log e)$$
$$h \geq n \log n - n \log e$$

Ordenação por comparação (árvore de decisão)



$$\#folhas \geq n!$$
$$\#folhas = 2^h$$
$$2^h \geq n!$$
$$\log 2^h \geq \log n!$$
$$h \geq \log(n/e)^n$$
$$h \geq n \log(n/e)$$
$$h \geq n(\log n - \log e)$$
$$h \geq n \log n - n \log e$$
$$h \geq n \log n$$

Ordenação por comparação (árvore de decisão)



$$\#folhas \geq n!$$
$$\#folhas = 2^h$$
$$2^h \geq n!$$
$$\log 2^h \geq \log n!$$
$$h \geq \log(n/e)^n$$
$$h \geq n \log(n/e)$$
$$h \geq n(\log n - \log e)$$
$$h \geq n \log n - n \log e$$
$$h \geq n \log n$$
$$\Rightarrow h = \Omega(n \log n)$$

Ordenação sem comparações

Counting sort

Radix sort

Bucket sort

Counting sort

Counting sort

Contagem de frequências

Uso de memória extra

Saber faixa de valores a serem ordenados (1..k)

37, 825, 3, 248, ...

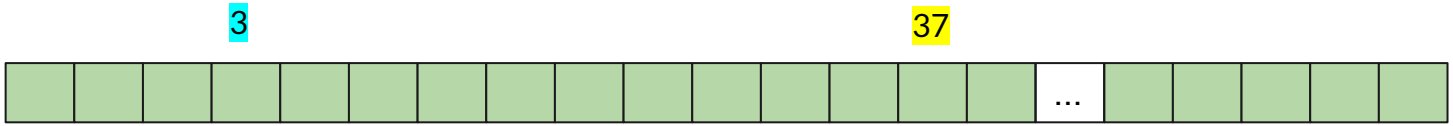
Counting sort

Contagem de frequências

Uso de memória extra

Saber faixa de valores a serem ordenados (1..k)

37, 825, 3, 248, ...



Counting sort (animação)

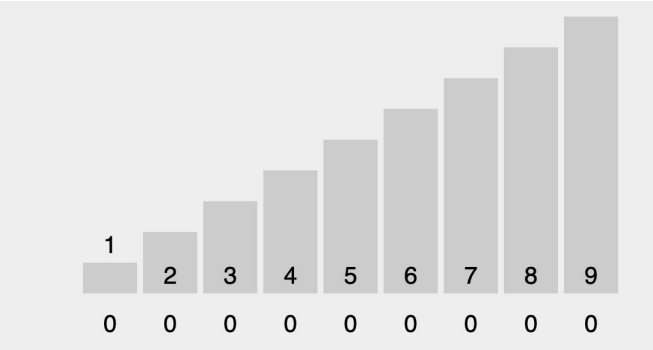
Counting sort (algoritmo)

[2, 3, 8, 7, 1, 2, 2, 2, 7, 3, 9]

```
COUNTING-SORT(A, B, k)
for i ← 1 to k
    do C[i] ← 0
for j ← 1 to length[A]
    do C[A[j]] ← C[A[j]] + 1
▷ C[i] now contains the number of elements equal to i.
for i ← 2 to k
    do C[i] ← C[i] + C[i − 1]
▷ C[i] now contains the number of elements less than or equal to i.
for j ← length[A] downto 1
    do B[C[A[j]]] ← A[j]
       C[A[j]] ← C[A[j]] − 1
```

Counting sort (algoritmo)

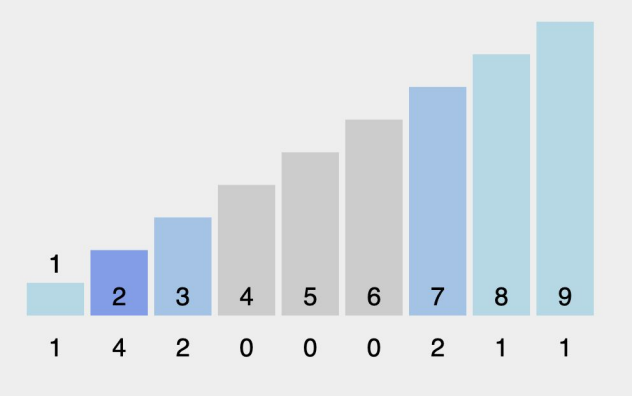
[2, 3, 8, 7, 1, 2, 2, 2, 7, 3, 9]




```
COUNTING-SORT(A, B, k)
for i ← 1 to k
    do C[i] ← 0
for j ← 1 to length[A]
    do C[A[j]] ← C[A[j]] + 1
▷ C[i] now contains the number of elements equal to i.
for i ← 2 to k
    do C[i] ← C[i] + C[i − 1]
▷ C[i] now contains the number of elements less than or equal to i.
for j ← length[A] downto 1
    do B[C[A[j]]] ← A[j]
       C[A[j]] ← C[A[j]] − 1
```

Counting sort (algoritmo)

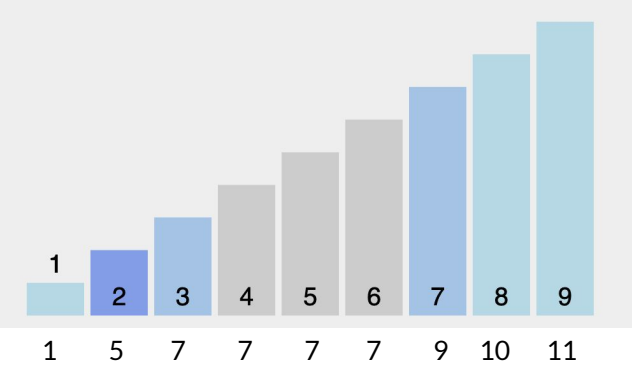
[2, 3, 8, 7, 1, 2, 2, 2, 7, 3, 9]




```
COUNTING-SORT(A, B, k)
for i ← 1 to k
    do C[i] ← 0
for j ← 1 to length[A]
    do C[A[j]] ← C[A[j]] + 1
▷ C[i] now contains the number of elements equal to i.
for i ← 2 to k
    do C[i] ← C[i] + C[i − 1]
▷ C[i] now contains the number of elements less than or equal to i.
for j ← length[A] downto 1
    do B[C[A[j]]] ← A[j]
       C[A[j]] ← C[A[j]] − 1
```

Counting sort (algoritmo)

[2, 3, 8, 7, 1, 2, 2, 2, 7, 3, 9]

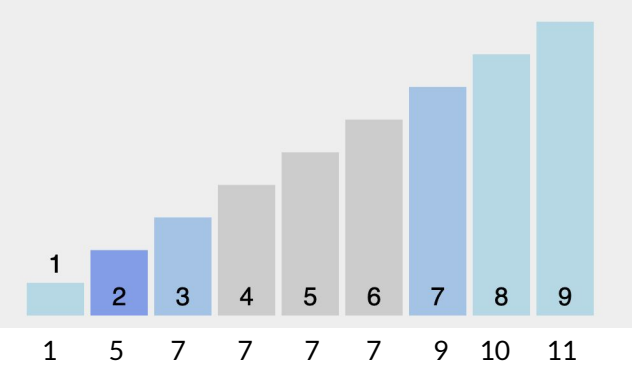


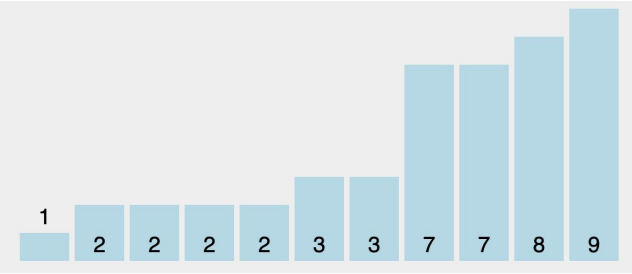

```
COUNTING-SORT(A, B, k)
for i ← 1 to k
    do C[i] ← 0
for j ← 1 to length[A]
    do C[A[j]] ← C[A[j]] + 1
▷ C[i] now contains the number of elements equal to i.
for i ← 2 to k
    do C[i] ← C[i] + C[i − 1]
▷ C[i] now contains the number of elements less than or equal to i.
for j ← length[A] downto 1
    do B[C[A[j]]] ← A[j]
       C[A[j]] ← C[A[j]] − 1
```

Counting sort (algoritmo)

```
COUNTING-SORT(A, B, k)
for i ← 1 to k
    do C[i] ← 0
for j ← 1 to length[A]
    do C[A[j]] ← C[A[j]] + 1
▷ C[i] now contains the number of elements equal to i.
for i ← 2 to k
    do C[i] ← C[i] + C[i − 1]
▷ C[i] now contains the number of elements less than or equal to
for j ← length[A] downto 1
    do B[C[A[j]]] ← A[j]
       C[A[j]] ← C[A[j]] − 1
```

[2, 3, 8, 7, 1, 2, 2, 2, 7, 3, 9]





Counting sort (execução)

https://youtu.be/3bm7NgKJpj4

Counting sort (análise de tempo)

```
COUNTING-SORT(A, B, k)
for i ← 1 to k
    do C[i] ← 0
for j ← 1 to length[A]
    do C[A[j]] ← C[A[j]] + 1
▷ C[i] now contains the number of elements equal to i.
for i ← 2 to k
    do C[i] ← C[i] + C[i − 1]
▷ C[i] now contains the number of elements less than or equal to i.
for j ← length[A] downto 1
    do B[C[A[j]]] ← A[j]
       C[A[j]] ← C[A[j]] − 1
```

Counting sort (análise de tempo)

```
COUNTING-SORT(A, B, k)
for i ← 1 to k
    do C[i] ← 0
for j ← 1 to length[A]
    do C[A[j]] ← C[A[j]] + 1
▷ C[i] now contains the number of elements equal to i.
for i ← 2 to k
    do C[i] ← C[i] + C[i − 1]
▷ C[i] now contains the number of elements less than or equal to i.
for j ← length[A] downto 1
    do B[C[A[j]]] ← A[j]
       C[A[j]] ← C[A[j]] − 1
```

$\boldsymbol{\theta}(k)$

Counting sort (análise de tempo)

```
COUNTING-SORT(A, B, k)
for i ← 1 to k                                          θ(k)
    do C[i] ← 0
for j ← 1 to length[A]                                  θ(n)
    do C[A[j]] ← C[A[j]] + 1
▷ C[i] now contains the number of elements equal to i.
for i ← 2 to k
    do C[i] ← C[i] + C[i − 1]
▷ C[i] now contains the number of elements less than or equal to i.
for j ← length[A] downto 1
    do B[C[A[j]]] ← A[j]
       C[A[j]] ← C[A[j]] − 1
```

Counting sort (análise de tempo)

```
COUNTING-SORT(A, B, k)
for i ← 1 to k                                          θ(k)
    do C[i] ← 0
for j ← 1 to length[A]                                  θ(n)
    do C[A[j]] ← C[A[j]] + 1
▷ C[i] now contains the number of elements equal to i.
for i ← 2 to k                                          θ(k)
    do C[i] ← C[i] + C[i − 1]
▷ C[i] now contains the number of elements less than or equal to i.
for j ← length[A] downto 1
    do B[C[A[j]]] ← A[j]
       C[A[j]] ← C[A[j]] − 1
```

Counting sort (análise de tempo)

```
COUNTING-SORT(A, B, k)
for i ← 1 to k                                          θ(k)
    do C[i] ← 0
for j ← 1 to length[A]                                  θ(n)
    do C[A[j]] ← C[A[j]] + 1
▷ C[i] now contains the number of elements equal to i.
for i ← 2 to k                                          θ(k)
    do C[i] ← C[i] + C[i − 1]
▷ C[i] now contains the number of elements less than or equal to i.
for j ← length[A] downto 1
    do B[C[A[j]]] ← A[j]                                θ(n)
       C[A[j]] ← C[A[j]] − 1
```

Counting sort (análise de tempo)

```
COUNTING-SORT(A, B, k)
for i ← 1 to k
    do C[i] ← 0
for j ← 1 to length[A]
    do C[A[j]] ← C[A[j]] + 1
▷ C[i] now contains the number of elements equal to i.
for i ← 2 to k
    do C[i] ← C[i] + C[i − 1]
▷ C[i] now contains the number of elements less than or equal to i.
for j ← length[A] downto 1
    do B[C[A[j]]] ← A[j]
       C[A[j]] ← C[A[j]] − 1
```

$\boldsymbol{\theta}(n + k)$

Counting sort (análise)

Estável?

In-place?

E se k >> n?

Counting sort (otimizações)

E se soubermos o mínimo e o máximo no array?

Pode-se aplicar a entradas com números negativos?

$$[x_1, x_2, x_{min}, \dots, x_k, x_{max}, \dots, x_m]$$

Counting sort (otimizações)

E se soubermos o mínimo e o máximo no array?

Pode-se aplicar a entradas com números negativos?

[10000, 10001, ... , 10010] (10 elementos)

Counting sort (otimizações)

E se soubermos o mínimo e o máximo no array?

Pode-se aplicar a entradas com números negativos?

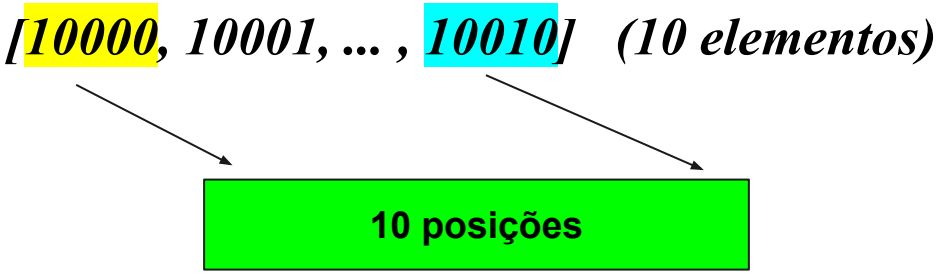
[10000, 10001, ... , 10010] (10 elementos)

10010 posições!!!!	10

Counting sort (otimizações)

E se soubermos o mínimo e o máximo no array?

Pode-se aplicar a entradas com números negativos?

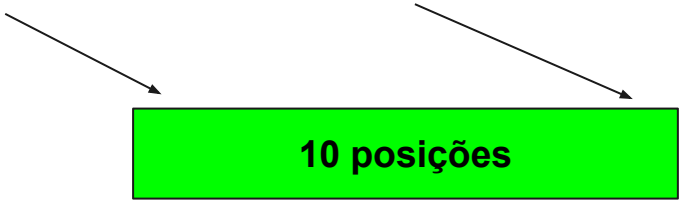


Counting sort (otimizações)

E se soubermos o mínimo e o máximo no array?

Pode-se aplicar a entradas com números negativos?

[10000, 10001, ... , 10010] (10 elementos)



diff = 10010 - 10000 = **10**

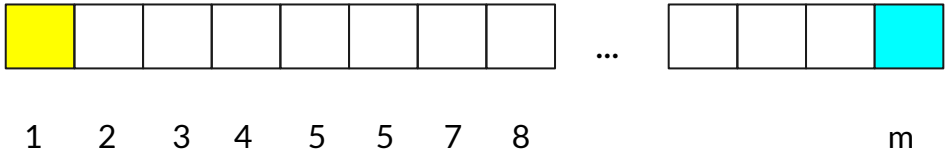
indice(x) = x - 10000

Counting sort (otimizações)

E se soubermos o mínimo e o máximo no array?

Pode-se aplicar a entradas com números negativos?

$$[x_1, x_2, x_{min}, \dots, x_k, x_{max}, \dots, x_m]$$



diff = $x_{max} - x_{min} = \mathbf{m}$

indice(x) = $x - x_{min}$

Radix sort

Radix sort

Similar a ordenação coluna a coluna do excel

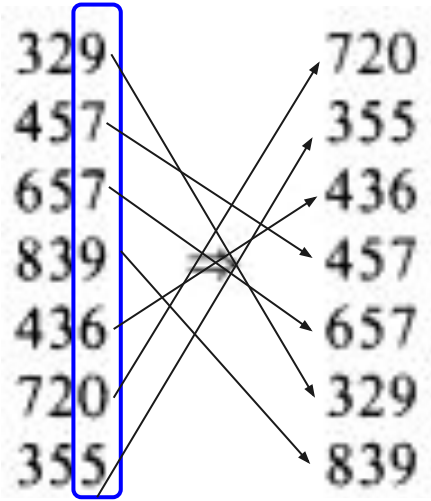
Usa ordenação dígito a dígito nos números (MSD, LSD)

Necessita de algoritmo auxiliar

Radix sort

329
457
657
839
436
720
355

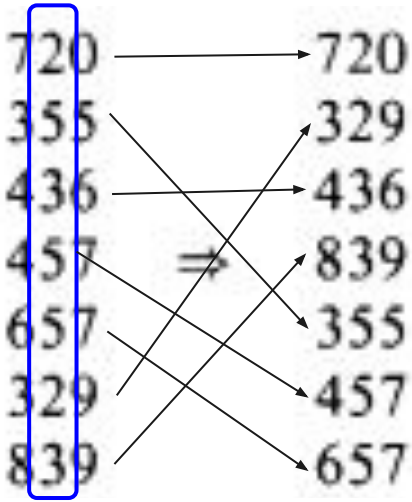
Radix sort



Radix sort

720
355
436
457
657
329
839

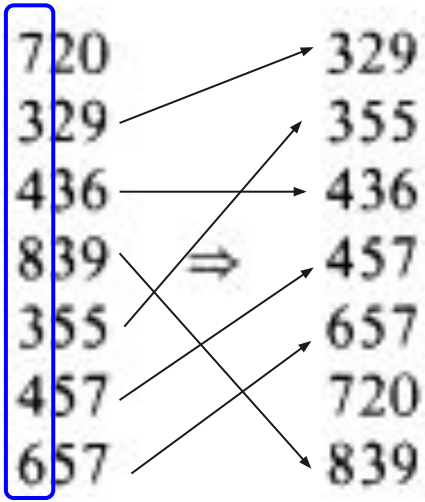
Radix sort



Radix sort

720
329
436
839
355
457
657

Radix sort



Radix sort (simulação)

Radix sort

```
radixSort( int [] A, int d) {
    for (i = 1 to d) //a partir do menos significativo
        ordene (stable) array A no dígito i
}
```

Radix sort (análise)

```
radixSort( int [] A, int d) {
   for (i = 1 to d) //a partir do menos significativo
      ordene (stable) array A no dígito i
}
```

θ (d*T(n))

Radix sort (análise)

```
radixSort( int [] A, int d) {
    for (i = 1 to d) //a partir do menos significativo
        counting sort em A no dígito i
}
```

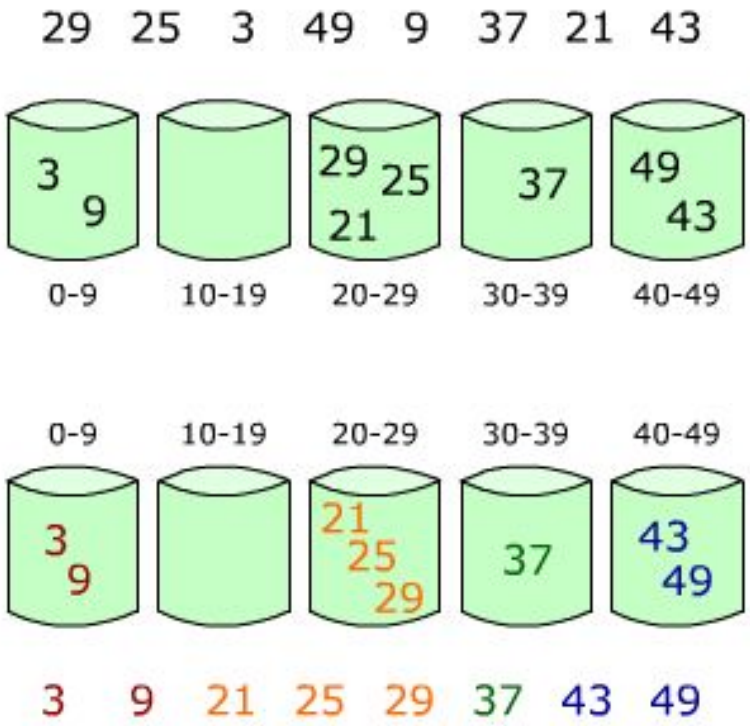
θ (d*(n + k))

⇩

θ (dn + dk)

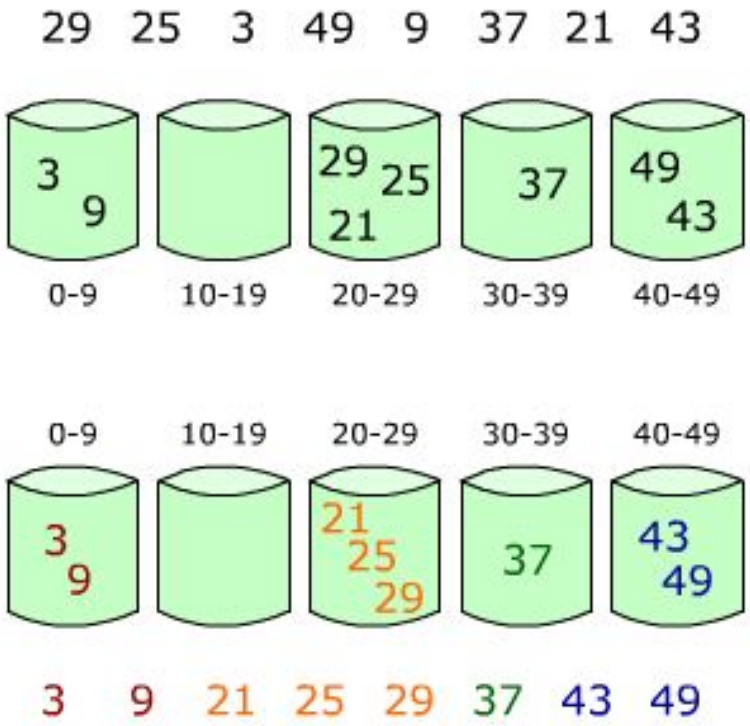
Bucket sort

Bucket sort


29 25 3 49 9 37 21 43
3 9
29 25 21
37
49 43
0-9 10-19 20-29 30-39 40-49
0-9 10-19 20-29 30-39 40-49
3 9
21 25 29
37
43 49
3 9 21 25 29 37 43 49

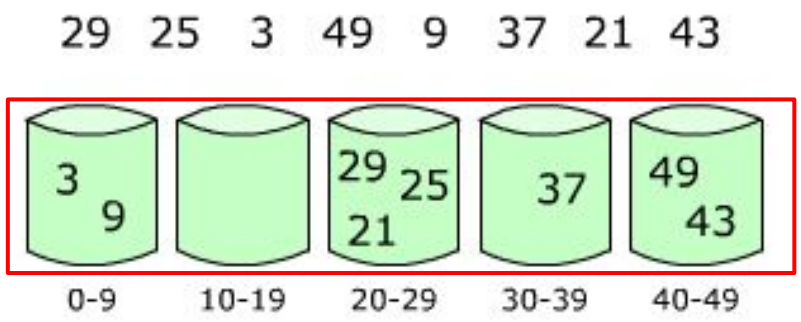
Bucket sort

```
function bucket-sort(array)
    buckets ← new array of 10 empty lists
    for i = 1 to array.length do
        insert array[i] into buckets[array[i]/10]
    for i = 0 to 9 do
        next-sort(buckets[i])
    return <buckets[0]>; ... ;<buckets[9]>
```

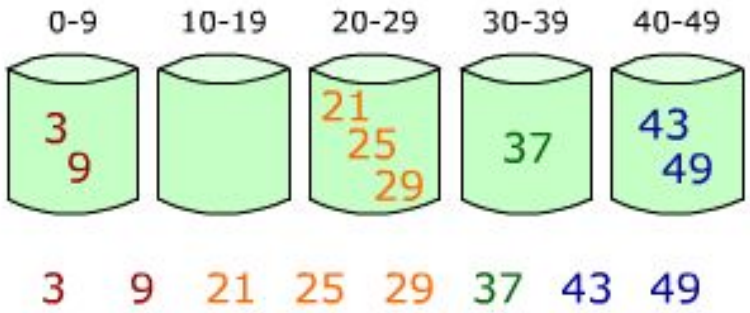


Bucket sort

```
function bucket-sort(array)
    buckets ← new array of 10 empty lists
    for i = 1 to array.length do
        insert array[i] into buckets[array[i]/10]
    for i = 0 to 9 do
        next-sort(buckets[i])
    return <buckets[0]>; ... ;<buckets[9]>
```

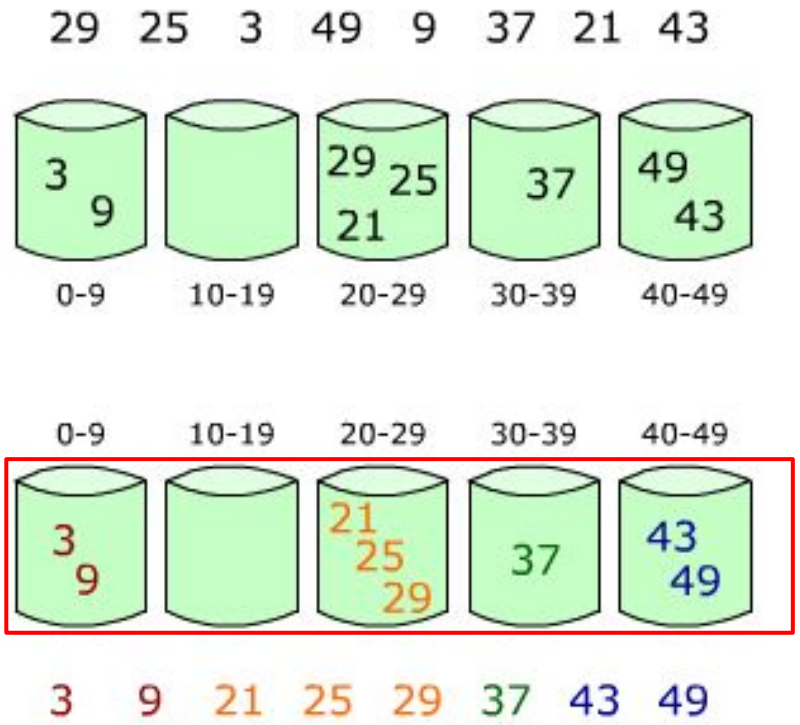


θ (n)



Bucket sort

```
function bucket-sort(array)
    buckets ← new array of 10 empty lists
    for i = 1 to array.length do
        insert array[i] into buckets[array[i]/10]
    for i = 0 to 9 do
        next-sort(buckets[i])
    return <buckets[0]>; ... ;<buckets[9]>
```

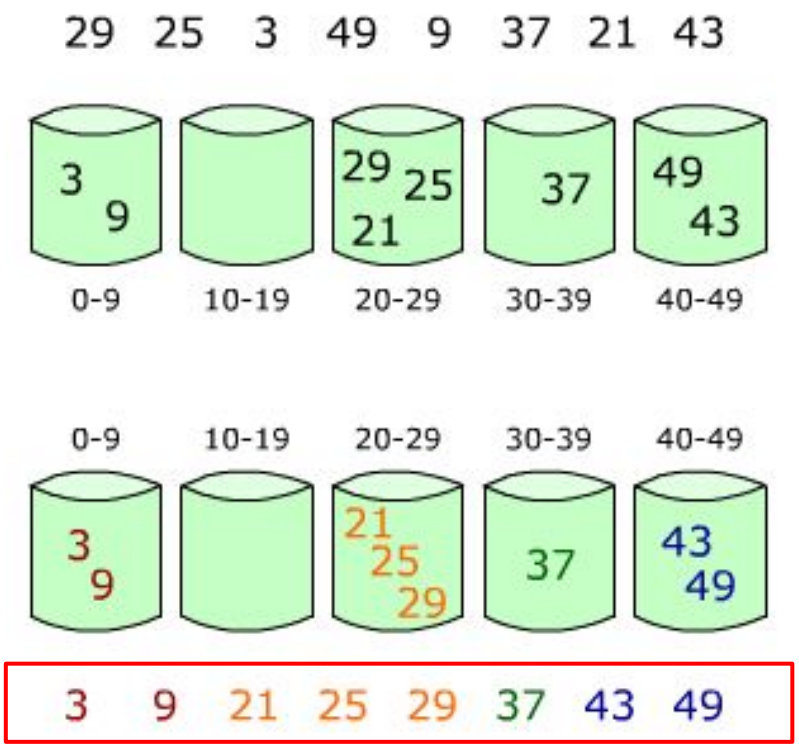


θ (n)

θ (b*T(n_b))

Bucket sort

```
function bucket-sort(array)
    buckets ← new array of 10 empty lists
    for i = 1 to array.length do
        insert array[i] into buckets[array[i]/10]
    for i = 0 to 9 do
        next-sort(buckets[i])
    return <buckets[0]>; ... ;<buckets[9]>
```



θ (n)

θ (b*T(n_b))

θ (b)

Bucket sort

```
function bucket-sort(array)
    buckets ← new array of 10 empty lists
    for i = 1 to array.length do
        insert array[i] into buckets[array[i]/10]
    for i = 0 to 9 do
        next-sort(buckets[i])
    return <buckets[0]>; ... ;<buckets[9]>
```

$\boldsymbol{\theta}$ (n + bf(n))

Bucket sort

```
function bucket-sort(array)
    buckets ← new array of 10 empty lists
    for i = 1 to array.length do
        insert array[i] into buckets[array[i]/10]
    for i = 0 to 9 do
        counting sort(buckets[i])
    return <buckets[0]>; ... ;<buckets[9]>
```

$\boldsymbol{\theta}$ (n + b(n+k))

Bucket sort

```
function bucket-sort(array)
    buckets ← new array of 10 empty lists
    for i = 1 to array.length do
        insert array[i] into buckets[array[i]/10]
    for i = 0 to 9 do
        counting sort(buckets[i])
    return <buckets[0]>; ... ;<buckets[9]>
```

$$\boldsymbol{\theta}\,(n + b(n+k))$$

$$\Downarrow \quad k = \boldsymbol{\theta}\,(n),\ b << n$$

$$\boldsymbol{\theta}\,(n + n) \Rightarrow \boldsymbol{\theta}\,(n)$$